Disney Paris Transfers

DEVELOPER HANDOFF

Design system, functionality & build directives

Everything needed to build, extend or take over the private-chauffeur booking site at disneyparistransfers.com — the exact palette, the pricing rules, the page inventory and the constraints that must not be broken.

Product	Stack	Languages	Document
disneyparistransfers.com	**Astro 7 · server-rendered HTML · TypeScript**	**9 · English default**	**v2.0**

CONTENTS

What is in this document

Sections 3 and 4 are the visual contract. Section 6 is the one to read twice — it is where money is decided, and it is the easiest part to get subtly wrong.

On the source of truth

The original visual prototypes live in `project/*.dc.html` and remain the authority for anything visual. Any divergence in colour, spacing or typography from those files is a bug, not a judgement call. This document restates them so you do not have to reverse-engineer the prototypes — but where the two disagree, the prototypes win.

SECTION 1

The product

A private-chauffeur (VTC) booking site for transfers between the Paris airports, central Paris and Disneyland Paris. One driver, one business — not a marketplace.

What the site does

1. **Quotes instantly.** A visitor picks a route, passenger count and vehicle and sees a real price immediately — not a "contact us for a quote" form.
2. **Captures the request.** The booking form stores the request, emails the driver and acknowledges the customer in their own language.
3. **Optionally takes payment.** Stripe Checkout, switchable on and off from the admin, full amount or a deposit.
4. **Sells in seven languages.** Each language has its own translated URLs, which is what earns local search ranking.

Who uses it

Audience	What they need from it
Travelling families The primary visitor	A fixed price they can trust before booking, child seats, flight tracking, and reassurance that a stranger will actually be at arrivals. Most arrive on mobile, often abroad, often in a hurry.
The driver The admin user	To see requests fast, reply in one tap via WhatsApp, and change prices without calling a developer. Not a technical user — the back office must stay blunt and obvious.

The commercial shape

Three revenue levers, all controlled by the driver at runtime with no deploy: the **rate grid** (per route, per passenger tier), **packages** (fixed-price bundles that bypass the grid) and **paid add-ons** (charged on top). A **night supplement** applies a percentage across the first two. Section 6 covers exactly how they compose.

Design intent worth preserving

The palette is warm and hand-made rather than corporate blue — cream, terracotta, gold. That is deliberate: the competition is faceless airport-transfer aggregators, and the whole positioning is "a real person will meet you". Keep it warm. Do not sanitise it into a generic SaaS look.

SECTION 2

Stack & hosting

Layer	Choice
Framework	Astro 7, server output on the Node adapter — no React, no client framework. TypeScript in strict mode
Client JS	A few vanilla TypeScript scripts in `src/scripts/` (menus, calculator, booking form) — about 10 kB in total
Styling	Tailwind CSS v4, CSS-based config in `src/styles/globals.css`
Fonts	Self-hosted `.woff2` in `public/fonts/`, no request to Google
Database	SQLite via `better-sqlite3`, single file at `data/app.db`
Email	Plain SMTP via `nodemailer` — the host's own mailbox, no third party
Payment	Stripe Checkout, switchable on/off from the admin
Admin auth	HMAC-signed session cookie + `scrypt` password. No auth library
Validation	`zod` on every API input, without exception

Hosting: this is not a serverless app

Target a **classic Node server** — OVH, Gandi, a VPS. SQLite and SMTP both assume a persistent filesystem and a long-lived process. Deploying to Vercel, Netlify or Cloudflare Pages would silently discard every booking on each restart: the app would appear to work and would quietly lose the business. If the project ever needs to move to a serverless platform, the database has to move to a hosted engine first.

Deliberate absences

There is no auth library, no ORM, no state-management library, no component library and no analytics. Each was considered and rejected: the site has one admin user, five tables and fewer than twenty pages. Please do not add one without a concrete problem it solves — the dependency surface is a feature here, and the client maintains this for years, not months.

Rendering strategy

- Every page is rendered on the server as complete HTML: title, description, canonical, `hreflang` × 9, Open Graph, JSON-LD and the full content — including the calculator's default price and every fare on the prices page. Nothing is left for a crawler to execute.
- Public pages are kept in an in-process cache (`src/lib/cache.ts`) and answer from memory like static files. Pages with a query string (the booking page) are never cached.

- Admin writes call `invalidatePages()`, so a price change is live immediately. The hourly expiry is a safety net for a missed invalidation, not the main mechanism.
- Browser scripts only enhance what is already in the HTML: the calculator, the destination picker, the booking form's steps and live estimate, the menus. One script per widget, in `src/scripts/`.

SECTION 3

Colour system

Twenty tokens, taken verbatim from the prototypes. They are defined once in `src/styles/globals.css` under `@theme` and consumed as Tailwind utilities. **Do not introduce a colour that is not on this page.**

Surfaces

Swatch	Hex	Utility	Use
	#FBF6ED	`bg-cream`	Page background — the default ground everywhere
	#F6E9D8	`bg-sand`	Section background, to separate bands of content
	#FFFDF9	`bg-surface`	Cards and the header — sits above cream
	#EBDFC9	`border-line`	Standard border on cards, fields, dividers
	#F0E2CC	`border-line-strong`	Table row separators — lighter than it sounds
	#E7D5B8	`bg-stripe`	The second tone in photo-placeholder hatching

Brand

Swatch	Hex	Utility	Use
	#B4552D	`text-brand` / `bg-brand`	Terracotta. Primary accent, primary CTA, prices, focus ring
	#8F3F1E	`bg-brand-dark`	Hover state of every terracotta button
	#D9A441	`bg-gold`	CTA on a brown ground only — never on cream
	#C4902F	`bg-gold-dark`	Hover state of gold

Text

Swatch	Hex	Utility	Use
	#3A2E24	`text-ink` / `bg-ink`	Headings and body. Also the footer and dark CTA ground
	#524334	`bg-ink-light`	Hover on a brown ground

	#6B5641	`text-ink-soft`	Running text, lead paragraphs
	#8F7455	`text-ink-mute`	Tertiary text, metadata, small print
	#9B8468	`text-ink-faint`	Captions and disclaimers only
	#C9B79E	`text-cream-soft`	Body text on a brown ground

WhatsApp & success

Swatch	Hex	Utility	Use
	#E8F0DC	`bg-whatsapp`	WhatsApp CTA on light ground; success banners
	#D8E5C5	`bg-whatsapp-dark`	Its hover state
	#B7CC96	`border-success-border`	Border of a success banner
	#4A6327	`text-success-text`	Text inside a success banner

Two rules about colour

Gold is for brown grounds only. Gold on cream fails contrast and looks cheap; terracotta is the accent on light, gold is the accent on dark.

Prices are always terracotta. Across the calculator, rate tables, route cards and booking estimate, the number the customer cares about is `text-brand`. Do not restyle prices per context.

SECTION 4

Typography, shape & layout

Two typefaces, no exceptions

Role	Family	Rules
Headings font-display	Bree Serif, 400	Every `h1` – `h3`, prices in the calculator, and the wordmark. One weight only — Bree Serif ships a single weight, so never fake a bold.
Everything else font-sans	Nunito 400 / 600 / 700 / 800	Body, labels, buttons, tables, navigation. 800 for buttons and emphasis, 700 for labels, 400 for running text.

Fonts are self-hosted

Both are served from `public/fonts/` (Google's own files under the Open Font License, one `.woff2` per script; Nunito is the variable font) and declared with `@font-face` at the top of `src/styles/globals.css`, with metric-matched fallback faces so nothing shifts while they load. **Never add a `<link>` to Google Fonts** — it reintroduces the layout shift and the third-party request that self-hosting exists to remove. Nunito is subset to `latin`, `latin-ext`, `cyrillic` and `vietnamese`; Chinese and Japanese deliberately fall back to system fonts, because shipping full CJK subsets would cost several megabytes.

Scale in practice

Transfers to Disneyland

Page h1 — `text-[34px]` mobile, `sm:text-[46px]` desktop

Most requested routes

Section h2 — `text-[32px]`

CDG Airport ↔ Disneyland Paris

Card title — `text-xl` / `text-[19px]`

Fixed fare agreed before you travel, flight tracking included, and a driver waiting with a sign in arrivals.

Lead paragraph — `text-[18px] leading-[1.7] text-ink-soft`

Shape

Element	Radius	Note

Cards	`rounded-[18px]` → `rounded-3xl`	18px for small cards, 24px for hero and feature panels
Buttons	`rounded-full`	Always fully round. Never a rounded rectangle
Form fields	`rounded-[10px]`	Defined once in the `.field` class
Pills / badges	`rounded-full`	Status chips, tier labels, filters

Elevation

Token	Value	Use
`shadow-card`	`0 12PX 24PX -14PX RGB(58 46 36 / .3)`	Card hover
`shadow-lifted`	`0 24PX 48PX -20PX RGB(58 46 36 / .35)`	The hero price calculator
`shadow-popover`	`0 16PX 32PX -12PX RGB(58 46 36 / .35)`	Phone menu, dropdowns

All three are brown-tinted rather than neutral black. That is what keeps the warmth — a grey shadow on cream reads as dirt.

Layout grid

- Content width `max-w-[1200px] mx-auto px-6` throughout.
- Section rhythm `py-[72px]` desktop, `py-14` mobile.
- The prototypes were drawn at **1280 px only**. All responsive behaviour was added during implementation and is not covered by them — you have latitude there, and you own the bugs.
- Three-column grids collapse to one below `md`. The header switches to a burger menu below `lg`.

SECTION 5

Component directives

Buttons

Variant	Appearance	When
Primary	`bg-brand`, `text-surface`, hover `bg-brand-dark`	The single most important action on the page. One per view
Outline	`border-2 border-brand`, `bg-surface`, `text-brand`, hover `bg-sand`	The secondary action beside a primary
Gold	`bg-gold`, `text-ink`, hover `bg-gold-dark`	Primary action inside a dark brown band
Gold outline	`border-2 border-gold`, transparent, `text-gold`	Secondary action inside a dark brown band
WhatsApp	`bg-whatsapp`, `text-ink`, hover `bg-whatsapp-dark`	Anything that opens WhatsApp. Always carries the 💬 glyph

Padding runs `px-6 py-3` to `px-7 py-3.5` depending on prominence; labels are `font-extrabold` at 15–17px. Disabled state is `disabled:opacity-60` and nothing else — no colour change.

Cards

`bg-surface` on a `border-line` border, radius 18–24px, padding `p-6`. Interactive cards get `hover:border-brand hover:shadow-card` and nothing more — no lift transform, no scale. The one exception is the hero calculator, which carries `shadow-lifted` permanently because it must read as the focal object of the page.

Form fields

Every input, select and textarea uses the shared `.field` class rather than per-component utilities: 1px `--color-line` border, 10px radius, cream ground, 12px padding, 15px text. Focus is a 2px terracotta outline inset by 1px. Labels use `.field-label` — a flex column, 700 weight, 14px.

Admin forms are plain HTML forms

Each admin form posts to its own page. A successful save answers with a redirect (`?saved=…`), so a browser refresh never re-submits; a failed save re-renders the page with the values the driver typed and the error next to that form (`src/lib/admin/page.ts`). Astro's built-in CSRF check requires a matching `Origin` header on form posts — browsers always send it. Keep it that way.

Photo placeholders

Real photography does not exist yet. Every image slot renders a `.photo-placeholder` — 45° hatching between `--color-line-strong` and `--color-stripe` — carrying a text description of the intended shot. They are meant to be visibly unfinished, so nobody ships them by accident. Replacing them is a mechanical job; the descriptions tell you what to source.

Status colour, used consistently

State	Treatment	Where
New / active	`bg-sand text-brand`	Booking status, active filter
Confirmed / success	`bg-whatsapp text-success-text`	Confirmed booking, success banner
Quoted / neutral	`bg-cream text-ink-soft`	Intermediate states
Cancelled / inert	`bg-line text-ink-mute`	Cancelled booking

SECTION 6

Pricing engine

Three layers, all editable by the driver at runtime with no deploy. Read this section before changing anything that touches a number.

```
  base      rate grid × vehicle multiplier × trip
            (or a package's flat price, which replaces all of it)
+ night     base × night %, if pickup time falls inside night hours
+ extras    the paid add-ons chosen — never night-surcharged
= quoted_price_cents
```

Layer 1 — the rate grid

- 20 connections between 8 zones: CDG, Orly, Beauvais, Disneyland, Paris, Versailles, La Défense, Val d'Europe.
- Each holds **six one-way prices in whole euros**, one per passenger tier: `1–3, 4, 5, 6, 7, 8`.
- The grid is **symmetric** — `cdg-disney` also serves Disney → CDG. One row per connection, never two.
- Round trip is simply ×2.
- A connection **missing** from the grid means "on request, reply within 2 h". The site must never invent a price for it.

Vehicle multipliers

Vehicle	Max pax	Bags	Multiplier
Saloon id `saloon`	4	3	×1
SUV	4	4	×1.1
Van	8	8	×1
Premium Mercedes	3	3	×1.5

The id `saloon` was renamed from `berline` when the codebase moved to English. A guarded migration rewrites old rows, and both the admin and the email templates fall back to the raw id rather than crashing on a vehicle they do not recognise.

Layer 2 — the night supplement

One global window plus one percentage, both set in the admin. It applies to fares, routes and any package flagged for it — **never to add-ons**.

Behaviour	Rule
Wrapping past midnight	A window whose end is at or before its start wraps — `22:00 → 05:30` covers both the evening and the early morning. This is the normal case, not an edge case.

Boundaries	Start is **inclusive**, end is **exclusive**. A 22:00–05:30 window surcharges a 05:29 pickup and not a 05:30 one.
No time supplied	The supplement never applies. There is nothing to judge it by, so those go out to be quoted by hand.
Round trips	Applied per leg *before* doubling, so both legs carry it — based on the outbound pickup time, since the form collects only one.

Worked examples — an 88 € fare at +35 %, window 22:00–05:30

Pickup	Price	Why
14:00	88.00 €	Outside the window
21:59	88.00 €	One minute before it opens
22:00	119.00 €	Start is inclusive
03:00	119.00 €	The window wraps past midnight
05:29	119.00 €	Last minute inside
05:30	88.00 €	End is exclusive
23:30, round trip	238.00 €	119 × 2 — surcharge on both legs

Layer 3 — packages and add-ons

	Package	Paid add-on
Replaces or adds?	Replaces the fare entirely — ignores the grid, the vehicle multiplier and the round-trip doubling	Charged on top of whatever the fare came to
Night-surcharged?	Optional, per package	**Never.** A child seat costs the same at 3 am as at 3 pm
Quantity	One per booking	Flat fee, or per-unit up to a cap
Example	"CDG ⇄ Disneyland round trip, up to 6 passengers — 150 €"	"Child seat — 12 € each, max 3"

Package and add-on names are not translated

They are typed by the driver at runtime and shown **verbatim in all seven languages** — you cannot translate a string that does not exist at build time. Only the wording *around* them (headings, "Up to {pax} passengers", the night-rate note) goes through `dict.pricing.*`. If the client later wants translated package names, that needs a per-language name field; it is not a bug to be patched.

Two invariants

1. The server always recalculates

The amount the browser sends is never used, for storage or for charging. `serverQuote()` in `src/lib/booking.ts` decides the price from the database, every time. The booking form **deliberately mirrors** that logic client-side so the visitor sees the figure that will actually be stored — if you change one, change the other, or the on-screen estimate and the confirmation email will disagree.

2. A booking freezes its own breakdown

Each booking stores `base_price_cents`, `night_surcharge_cents`, `extras_price_cents`, `package_name` and `extras_json` — labels and unit prices *copied*, not referenced. Editing a package or add-on later must never rewrite what an existing customer was quoted.

`quoteBreakdown()` and `applyNight()` in `src/lib/prices.ts`, and `priceExtras()` in `src/lib/catalog.ts`, are pure and client-safe by design — they must never import the database or a `server-only` module.

SECTION 7

Pages & routing

Seven languages, English default

EN default FR ES IT RU ZH JA

Every public URL is language-prefixed. `middleware.ts` redirects an unprefixed URL using the visitor's cookie, then `Accept-Language`, then English — with a **307**, not a 301, because the choice depends on the visitor's headers and must not be cached as permanent.

URL segments are translated — this is the SEO engine

Page	EN	FR	ES	IT	RU	ZH / JA
Home	no segment — `/en`, `/fr`, …					
Routes	transfers	trajets	traslados	trasferimenti	transfery	transfers
Prices	prices	tarifs	precios	prezzi	tseny	prices
Booking	booking	reservation	reserva	prenotazione	bronirovanie	booking
About	about	a-propos	sobre-nosotros	chi-siamo	o-nas	about
FAQ	faq	faq	preguntas-frecuentes	domande-frequenti	voprosy	faq
Contact	contact	contact	contacto	contatti	kontakty	contact

`SEGMENTS` in `src/lib/i18n/routes.ts` is the single source of truth. Any language/segment combination that does not exist returns **404 deliberately**, so Google never sees two URLs for the same content.

Only two page files

`src/pages/[locale]/index.astro` renders the home page. Everything else goes through `src/pages/[locale]/[...segments].astro`, which maps the URL segment back to a page key. This indirection exists because translated segments cannot be expressed as a folder structure — `/en/prices`, `/fr/tarifs` and `/es/precios` are one page.

To add a page

1. Add its key to `PAGE_KEYS` and its 9 segments to `SEGMENTS`.
2. Add its copy to the `Dictionary` type, then to all 7 dictionaries — the build stays broken until every language has it, which is intentional.
3. Create the server component under `src/components/pages/`.
4. Wire it into the `switch` and into `generateStaticParams`.

Route pages — one per priced connection

Eight dedicated SEO landing pages, each with its own duration, distance, rate table, FAQ and JSON-LD. Route *slugs* are **not** translated: they are proper nouns, and keeping them stable means one key per priced connection instead of seven.

Slug	Duration	Distance	Home page
`cdg-disneyland`	45 min	60 km	Featured
`orly-disneyland`	50 min	50 km	Featured
`beauvais-disneyland`	90 min	120 km	Featured
`paris-disneyland`	45 min	45 km	Featured
`cdg-paris`	40 min	35 km	Featured
`orly-paris`	35 min	25 km	—
`paris-beauvais`	75 min	85 km	—
`paris-versailles`	40 min	25 km	—

SECTION 8

Booking & payment flow

The customer journey

1. **Calculator, home page hero.** Route, passengers, trip type, vehicle — and pickup time, but only when the night supplement is switched on. Live price per vehicle; those too small for the group grey themselves out. Two exits: WhatsApp with a pre-filled message, or through to the booking form with the simulation carried in the query string.
2. **Booking form.** Optional package selector at the top, then trip details, then optional paid add-ons, then contact details. A live breakdown panel shows fare, night supplement, each add-on and the total.
3. **Submission.** `POST /api/booking` validates with zod, recalculates the price server-side, stores the booking with its frozen breakdown, and fires two emails.
4. **Payment, if enabled.** A pay button appears on the confirmation panel. `POST /api/checkout` opens a Stripe Checkout session using *only* the reference — the amount comes from the database.
5. **Confirmation.** The Stripe webhook marks the booking paid and sends a receipt.

Emails

Email	Language	Contents
Driver notification	Always English	Full trip detail, the price breakdown, and contact details. Reply-To is the customer, so a plain reply reaches them
Customer acknowledgement	Their own	Reference, trip summary, breakdown line by line
Payment receipt	Their own	Reference and amount. A copy goes to the driver

Four things that must not change

The webhook is the authority on payment, not the browser redirect. A customer who closes the tab after paying must still end up marked paid.

Email failure must never fail a booking. Sending is fire-and-forget behind `Promise.allSettled`: the request is already stored and visible in the admin even if SMTP is down.

The honeypot answers 200. The `website` field is accepted by the schema and silently discarded, so a bot learns nothing from the response.

Rate limiting is in-process and keyed on `x-forwarded-for` — 5 booking requests per IP per 10 minutes. It resets on restart, which is fine for spam but is *not* a security boundary. Behind several Node processes, move it to a shared store.

Without SMTP configured

Mail falls back to console logging. The site stays fully usable in development and bookings are still persisted — do not "fix" this by throwing.

SECTION 9

Admin back office

At `/admin`, English only and never translated. Excluded from the middleware's locale redirect and page cache, and `robots: noindex`. Session is a 12-hour HMAC-signed cookie scoped to `/admin`; the password is `scrypt`-hashed in an env var.

Screen	Controls
Bookings `/admin`	Every request, newest first, filterable by status. On a booking: change status, correct the price, write internal notes, reply via WhatsApp / phone / email. Also shows **how the price was calculated** — fare or package, night supplement, each add-on, total.
Rates `/admin/rates`	The grid, as six space-separated prices per connection. Clearing a line removes the connection ("on request"); a form at the bottom adds one. A banner states the current night rule, because **daytime prices go in here** and the supplement is added on top.
Packages & add-ons `/admin/packages`	Full CRUD on both. Packages take a name, description, price, passenger cap and a night-supplement flag. Add-ons take a label, price, per-unit flag and quantity cap. Either can be deactivated rather than deleted.
Settings `/admin/settings`	Night hours and percentage, with a live plain-language preview and a worked example. The Stripe switch and full/deposit mode. A read-only summary of the contact details, which live in `.env`.

Admin UX rules

- **Every change is live immediately**, in all 9 languages. Writes call `invalidatePages()`; there is no publish step and there should not be one.
- **Explain in plain words, not jargon.** The settings screen says "Pickups from 22:00 in the evening through to 05:30 the next morning are charged +35 %. An €80 transfer becomes 108 €." The driver is not a developer and should never have to reason about a wrapping time window.
- **Every action returns a visible result** — "Rate grid saved.", "Package created." Silence reads as failure.
- **Destructive actions confirm.** Deleting a package or add-on asks first.

> **Never expose a secret value**
>
> The settings screen reports whether SMTP and Stripe are configured, never the values. Any future admin surface must follow the same rule: report status, not secrets.

SECTION 10

Data model

Five tables in one SQLite file. The full annotated schema ships with the project at `delivery/schema.sql`.

Table	Holds
`bookings`	Every quote request and booking, with its frozen price breakdown, payment status and internal notes
`settings`	Key/value: the Stripe switch and mode, the night window and percentage
`rates`	The grid — one row per connection, prices as a JSON array of six whole euros
`packages`	Driver-created fixed-price offers
`extras`	Driver-created paid add-ons

Conventions

- **Money is in cents**, as integers, everywhere except the rate grid, which is in whole euros. Never floats.
- **Timestamps are ISO 8601 UTC** strings.
- **Slugs are generated** from names and de-duplicated with a numeric suffix — two packages may legitimately share a name.
- `prices.ts` seeds the grid on first boot only. Read the live grid with `getRates()`, never the `RATES` constant.

Migrations: SQLite has no ADD COLUMN IF NOT EXISTS

New columns must be added through the guarded helper in `src/lib/db.ts`, which checks `PRAGMA table_info` first. An existing `data/app.db` in production has to survive every upgrade — and it holds the entire business. Any deployment change that does not preserve that file loses every customer record.

Backups

The whole business is one file. It should be copied off the server on a schedule. This is the single highest-value operational task on the project and it is worth confirming it is actually running.

SECTION 11

Rules that must not be broken

Security

- Prices shown to a visitor are **always recalculated server-side** before anything is stored or charged.
- Stripe amounts come from `quoted_price_cents` in the database. The browser sends only a reference.
- `zod` validates every API input. No exceptions, including fields you believe are internal.
- No secret is ever committed, logged or returned by an endpoint.
- `ADMIN_PASSWORD_HASH` uses `:` as its separator, not `$` — dotenv expands `$name` and would silently truncate the hash.

Internationalisation

- Every visible string goes through the dictionary. Two exceptions only: the admin, and driver-typed package and add-on names.
- Adding a key to `Dictionary` breaks the build until all seven languages have it. That is the feature — do not weaken the type to unblock yourself.
- Phone, WhatsApp and email come from env vars, never hard-coded.
- Keep the disclaimer in all languages: *"Independent service, not affiliated with The Walt Disney Company."*

SEO

- Every public page sets `title`, `description`, `canonical` and `alternates`.
- `hreflang` plus `x-default` are generated for every page.
- The home page and route pages emit JSON-LD: `LocalBusiness`, `Service`, `FAQPage`.

Accessibility

- A visible focus ring — 2px terracotta, 2px offset — on every interactive element. Never remove it.
- A skip-to-content link opens every page.
- Live price regions carry `aria-live="polite"`; toggle buttons carry `aria-pressed`.
- `prefers-reduced-motion` is honoured globally.
- Wide tables scroll inside their own container; the page body never scrolls sideways.

Performance

- No client framework: a public page ships one stylesheet and at most three small scripts.
- Public pages are served from the in-process cache; admin writes invalidate it immediately.
- Only the booking page reads the query string, so the others are always cacheable.
- Fonts are preloaded and metric-matched fallbacks avoid layout shift.

SECTION 12

Environment & commands

Environment variables

Variable	Purpose
`APP_URL`	Public origin, used for canonicals and Stripe return URLs
`SESSION_SECRET`	Signs the admin session cookie. 32 characters minimum — required
`ADMIN_EMAIL`	The single admin login
`ADMIN_PASSWORD_HASH`	From `npm run admin:hash`. Paste unquoted
`SMTP_HOST` `SMTP_PORT` `SMTP_USER` `SMTP_PASS` `SMTP_SECURE`	Mailbox. Leave `SMTP_HOST` empty to log to console instead
`MAIL_FROM` `MAIL_TO`	Sender identity, and where requests are delivered
`STRIPE_SECRET_KEY` `STRIPE_WEBHOOK_SECRET`	Optional. Without them the admin's payment switch stays greyed out
`SITE_PHONE` `SITE_PHONE_DISPLAY` `SITE_WHATSAPP` `SITE_EMAIL`	Public contact details, shown on every page. Read at request time: edit, restart, no rebuild
`PORT` `HOST`	Optional. Where `npm start` listens (default 3000 on all interfaces)
`DATABASE_PATH`	Optional. Defaults to `./data/app.db`

Commands

Command	Does
`npm run dev`	Development server on :3000
`npm run build`	Production build → `dist/`
`npm start`	Production server (`node server.mjs`, reads `.env`)
`npm run lint`	ESLint
`npm run check`	`astro check` — TypeScript on `.astro` and `.ts` files
`npm run admin:hash -- 'password'`	Generates `ADMIN_PASSWORD_HASH`

Before shipping any change

`npm run check`, `npm run lint` and `npm run build` must all pass. There is no automated test suite — that is the honest state of the project, and it means the three commands above plus a manual pass through the booking flow are the whole safety net.

Project documentation

File	Audience
`CLAUDE.md`	Developers and coding agents — authoritative
`CLAUDE.fr.md`	French translation of the above, for the client
`delivery/INSTALLATION-GUIDE.md`	The client — install, deploy, operate
`delivery/GUIDE-INSTALLATION.md`	French version of the same
`delivery/schema.sql`	Annotated database schema, with useful queries
`project/*.dc.html`	The original visual prototypes — authoritative for design

SECTION 13

Known gaps

Stated plainly so they are decisions rather than surprises. None of these is a defect to be quietly patched — each is a choice waiting on the client.

Gap	Detail
Photography	Every image is a hatched placeholder carrying a description of the intended shot. Vehicle exteriors, an arrivals meet-and-greet, and a driver portrait are needed. Mechanical to replace once supplied.
Customer reviews	The three review cards on the home page are reserved placeholders, not real testimonials. They must be replaced with genuine ones — or removed — before launch. Shipping invented reviews would be both dishonest and, in the EU, unlawful.
Non-CDG fares	Only the CDG connections use rates observed on the market. Every other connection was estimated on the same model and **must be confirmed by the client** before going live.
Translation review	The non-French translations were written by an assistant and have not been checked by a native speaker. Russian, Chinese and Japanese in particular should be reviewed before launch — including the newer pricing copy.
No test suite	There are no automated tests. The pricing engine is the obvious place to start: the night-window boundaries and the composition of packages with add-ons are pure functions and cheap to cover.
Rate limiting is per-process	Fine for one Node process. Behind several, or behind a CDN that changes the client IP, it needs to move to the database or a shared store.
Package names are single-language	By design, explained in section 6. If the client wants them translated, it needs a per-language name field on the packages and extras tables plus an admin UI for it.
Legal pages	French law requires *mentions légales*, CGV and a privacy notice for a VTC operator collecting personal data. None exist yet. This is a launch blocker, and the content has to come from the client or their accountant.

If you read only one thing

Prices are recalculated server-side, always. The browser is never trusted with an amount.

`data/app.db` **is the entire business.** Preserve it through every deployment, and back it up.

The prototypes in `project/` **win on anything visual.**